

Tratamento de Erros em JavaScript com `try...catch`

Aprenda a construir aplicações que não travam — mesmo quando algo dá errado. Neste módulo, você vai entender como o JavaScript lida com erros em tempo de execução e como usar a estrutura `try...catch` para proteger seus usuários de experiências ruins.

DESENVOLVIMENTO DE SISTEMAS

JAVASCRIPT

TRATAMENTO DE ERROS



Agenda

O que vamos aprender hoje

01

Erros em tempo de execução

O que acontece quando o código quebra

03

Sintaxe e exemplos práticos

Código real com validação de dados

02

A analogia da rede de proteção

O conceito central do try...catch

04

O bloco finally e boas práticas

Como usar erros de forma profissional

O que acontece quando o código **quebra**?

Erros de **tempo de execução** (Runtime Errors) acontecem enquanto o programa está rodando — não antes. O código parece correto, mas algo inesperado ocorre: o banco de dados fica fora do ar, a internet cai, o usuário digita um valor inválido. O resultado? O aplicativo trava, a tela congela e o usuário fica frustrado.

Banco de dados fora do ar

A aplicação tenta buscar dados e não recebe resposta

Internet instável

Uma requisição falha no meio do caminho

Dado inválido

O usuário digita texto onde se espera um número

 O bom desenvolvedor **não ignora** a possibilidade de erros — ele a prevê e trata antes que chegue ao usuário.

A Analogia da Rede de Proteção

Pense no trapezista

O trapezista dá um salto — pode conseguir ou pode cair. A rede abaixo não impede o salto, mas garante que uma queda não seja fatal. É exatamente assim que o `try...catch` funciona no JavaScript.

- **try** = o salto (o código que pode dar errado)
- **catch** = a rede (o que fazer se algo falhar)

Como funciona o fluxo

O JavaScript executa o bloco `try` normalmente. Se tudo correr bem, o `catch` é completamente ignorado. Se algo falhar, o controle salta imediatamente para o `catch` — e o programa **não trava**.



✓ O usuário vê uma mensagem amigável em vez de uma tela branca de erro.

Sintaxe Básica e Primeiro Exemplo

A estrutura fundamental

O bloco `try` envolve o código que pode falhar. O `catch` recebe o objeto de erro e define o que fazer. Veja o exemplo ao lado: tentamos acessar uma propriedade de uma variável `null` — algo muito comum no dia a dia.



⚠️ Acessar propriedades de `null` ou `undefined` é uma das causas mais comuns de erros em JavaScript!

Código

```
// Sem try...catch → o programa TRAVA  
let usuario = null;  
console.log(usuario.nome); //
```

Entendendo o Objeto de Erro

Quando um erro é capturado pelo `catch`, o JavaScript entrega um **objeto** com informações valiosas sobre o que aconteceu. Você pode chamá-lo de `error`, e ou qualquer nome — mas "error" é a convenção mais usada.

`error.message`

A descrição do que deu errado. Ex: *"Cannot read properties of null"*. Use para debugar e para criar mensagens úteis ao usuário.

`error.name`

O tipo do erro. Ex: `TypeError`, `ReferenceError`, `SyntaxError`. Útil para tratar tipos diferentes de erros de formas diferentes.

`console.error()`

Use para registrar o erro técnico no console do navegador — para o **desenvolvedor**. Para o **usuário**, exiba sempre uma mensagem amigável na tela.

```
} catch (error) {  
  console.error(error.name + ": " + error.message); // Para o dev  
  alert("Ops! Não foi possível carregar os dados."); // Para o usuário  
}
```

Exemplo Prático: Validação de Dados

O cenário: divisão por zero

Imagine uma função de calculadora. Se o usuário digitar **zero como divisor**, o resultado seria Infinity — um valor inválido, mas que não causaria um erro.

A solução é **lançar um erro de propósito** com `throw new Error()` dentro do `try`, forçando o código a ir para o `catch`.



throw significa "lançar". Você lança um erro como uma exceção que precisa ser capturada.

Código

```
function dividir(a, b) {  
  try {  
    if (b === 0) {  
      throw new Error("Divisor não pode ser zero!");  
    }  
    let resultado = a / b;  
    console.log("Resultado: " + resultado);  
  } catch (error) {  
    console.error(error.message);  
    alert("Erro: " + error.message);  
  }  
}  
  
dividir(10, 2); //
```

O Bloco **finally**: A Limpeza da Bagunça

O **finally** é o bloco que roda **sempre** — não importa se o **try** funcionou ou se o **catch** foi acionado. Pense nele como o responsável pela "limpeza" após a operação: fechar conexões, esconder indicadores de carregamento, liberar recursos, recarregar a página ou algum elemento, disparar uma função, etc.

```
function carregarDados() {  
  mostrarSpinner(true); // Inicia o loading  
  try {  
    // Busca dados do servidor  
    let dados = buscarDoServidor();  
    exibirDados(dados);  
  } catch (error) {  
    alert("Falha ao carregar. Tente novamente.");  
  } finally {  
    mostrarSpinner(false); // SEMPRE para o loading  
  }  
}
```

Quando usar o **finally**?

- Fechar conexão com banco de dados
- Esconder o spinner de carregamento
- Liberar arquivos abertos
- Resetar estados da interface

✔ ☒ O spinner nunca fica preso na tela — independente do resultado da operação.

Boas Práticas para o Dia a Dia



Nunca deixe o catch vazio

Um `catch` vazio silencia o erro sem tratá-lo. Você perde rastreabilidade e cria bugs invisíveis que são difíceis de encontrar depois.



Envolva só o que pode falhar

Não coloque o arquivo inteiro dentro de um `try`. Isole apenas o trecho de código que realmente representa um risco de falha.



Mensagens úteis para o usuário

Prefira *"Verifique sua conexão com a internet"* a *"Error 500: NullPointerException"*.

Lembre-se que o usuário **não é um desenvolvedor**.

Erros Comuns em JavaScript

Conheça os Tipos de Erro mais frequentes

Tipo (error.name)	Quando acontece	Exemplo
<code>TypeError</code>	Operação em tipo incorreto	<code>null.nome</code>
<code>ReferenceError</code>	Variável não existe	<code>console.log(x)</code> sem declarar x
<code>SyntaxError</code>	Código mal escrito	Falta de fechamento de chave
<code>RangeError</code>	Valor fora do intervalo	Array com tamanho negativo
<code>Error</code> (customizado)	Lançado com <code>throw</code>	<code>throw new Error("Inválido")</code>



Você pode usar `if (error instanceof + tipo_do_erro)` dentro do `catch` para tratar cada tipo de erro de forma diferente.

Praticando

Desafio: Encontre o Problema

❌ Código com problemas

```
try {  
  let dados = obterUsuario();  
  console.log(dados.perfil.foto);  
  salvarNoBanco(dados);  
  enviarEmail(dados.email);  
  gerarRelatorio();  
  atualizarCache();  
} catch (e) {  
  // silêncio...  
}
```

❌ Quantos problemas você consegue identificar aqui?

✅ Versão melhorada

```
function carregarPerfil(id) {  
  try {  
    let dados = obterUsuario(id);  
    if (!dados) throw new Error("Usuário não encontrado");  
    return dados.perfil.foto;  
  } catch (error) {  
    console.error("Erro em carregarPerfil:", error.message);  
    mostrarMensagem("Não foi possível carregar o perfil.");  
    return null;  
  }  
}
```

✅ try focado + catch útil = código profissional

Recapitulando

O que aprendemos **hoje**

1 Erros de runtime são inevitáveis

Eles acontecem por fatores externos: internet, banco de dados, dados do usuário. Prever é mais profissional do que ignorar.

3 `throw` cria erros intencionais

Use `throw new Error()` para validar regras de negócio e forçar o tratamento adequado no `catch`.

2 `try...catch` protege sem travar

O `try` tenta executar; o `catch` captura a falha. O programa continua rodando e o usuário recebe uma mensagem útil.

4 `finally` garante a limpeza

Roda sempre — perfeito para fechar conexões, parar spinners e liberar recursos independente do resultado.

Próximos Passos

Agora que você domina o `try...catch`, está pronto para escrever aplicações mais robustas e profissionais. Pratique os exemplos desta aula e depois explore como integrar tratativa de erros com requisições assíncronas usando `async/await`.

Pratique agora

Reescreva o código antigo do seu projeto de API adicionando `try...catch` nos pontos de risco.

  **Dica final:** Um código que trata erros com cuidado não é sinal de desconfiança — é sinal de **maturidade profissional**.

